CIS-25 Midway Project

Personal Budget Tracker (C++ Console Application)

Ai Yin (Ivy) Yap | CIS-25 Programming Using C++ | Instructor: Lakisha Brooks | Fall 2026

What the Program Does

This program is a Personal Budget Tracker. It lets a user keep track of money spent on different things, like food, rent, or bills. Each entry is called a "record." A record has a date, a category (like "Food" or "Rent"), and an amount of money.

When the program starts, it shows a welcome screen and prints the size (in bytes) of the main data types used in the program. Then it asks the user to log in as either an **Admin** or a **Regular User**. An Admin can add and delete records. A Regular User can only view records, search for a record, and see the total summary. This keeps the data safer, because not everyone can change it.

After logging in, the user sees a menu with six choices: Add Record, View Record, Search Record, View Summary, Delete Record, and Exit. The program keeps asking for a choice until the user picks Exit. All records are loaded from a file called `personal_finance_data.csv` when the program starts, and saved back to that same file every time a record is added or deleted, so the data is not lost when the program closes.

Before saving a new record, the program checks the input carefully. If the date is not in the correct YYYY-MM-DD format, if the category is left empty, or if the amount is zero or negative, the program shows an error message and asks again until the input is valid. This stops bad data from ever being saved.

Concepts Used in This Program

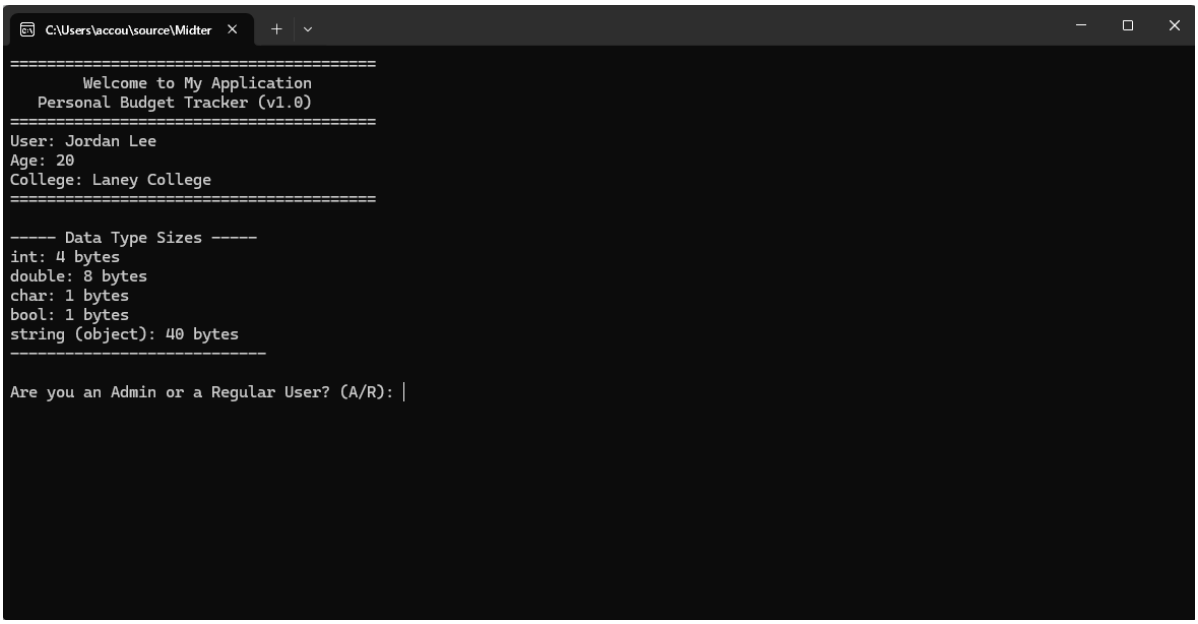
1. Data types and their sizes	The function `printDataSizes()` uses the `sizeof()` operator to print how many bytes an int, double, char, bool, and string take up. This is shown on the welcome screen (Screenshot 1).
2. Two or more classes	The program has a `Transaction` class that stores one record's date, category, and amount. It also has a `User` base class, with `Admin` and `RegularUser` as subclasses that inherit from it. This controls who is allowed to add or delete records.
3. Arrays	Records are stored using parallel arrays: `transactionID[]`, `transactionDate[]`, `category[]`, and `amount[]`. Each index across the four arrays belongs to the same record.
4. Pointers	In View Record (Screenshot 4), the program uses a pointer (`double *amountPtr = &amount[0];`) to read the first transaction's amount through the pointer instead of the array directly.

5. Binary search	The function `binarySearchByID()` searches the sorted array of record IDs to quickly find a record. It is used both in Search Record (Screenshot 5) and Delete Record (Screenshot 7).
6. Strings	Strings are used everywhere in the program: the date, the category, and the user's name are all stored as `string` values, and are checked and compared during input validation.
7. File types and file I/O	The program reads from and writes to a CSV (Comma-Separated Values) file, `personal_finance_data.csv`. `loadRecords()` uses an `ifstream` to read the file when the program starts, and `saveRecords()` uses an `ofstream` to write the updated records back to the file after every add or delete, so changes are permanently saved.

Program Screenshots

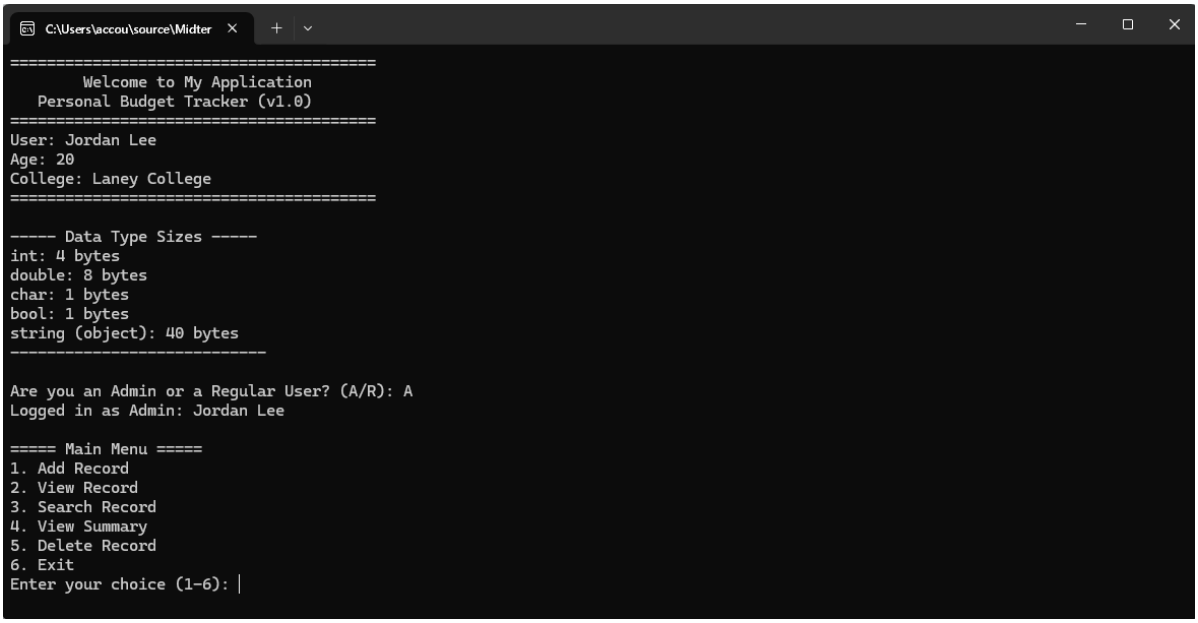
Below are screenshots of the program running from start to finish, logged in as Admin, so that every menu option could be demonstrated.

Screenshot 1 — Welcome Screen and Data Type Sizes



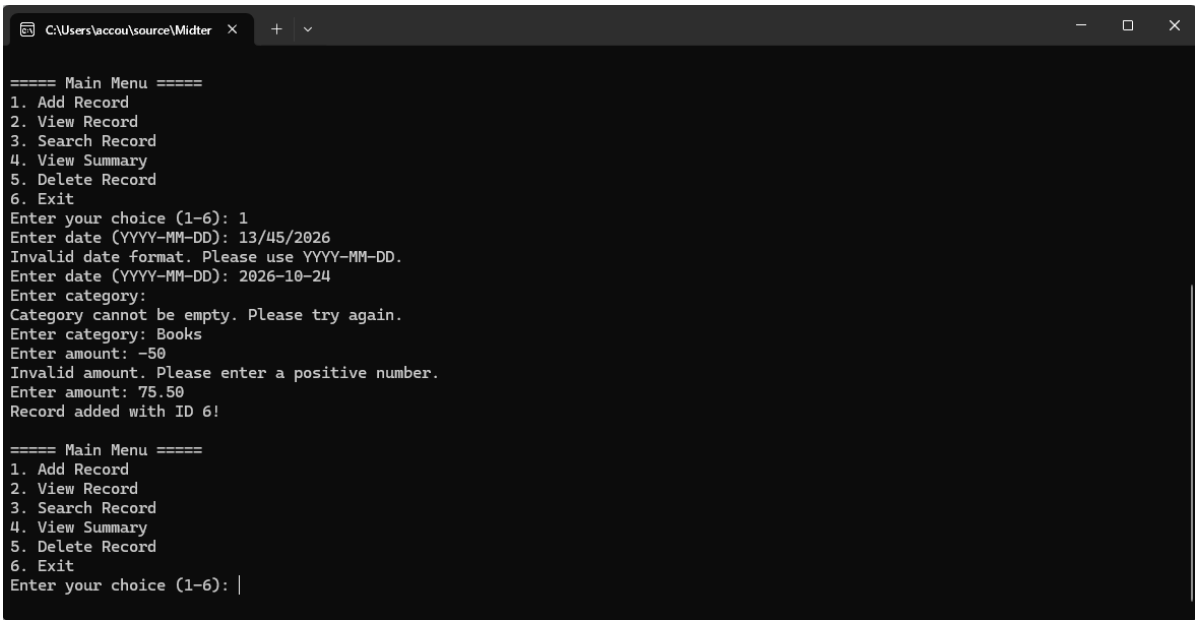
The program starts, prints the welcome banner, and shows the size of each data type using sizeof(). Then it asks whether the user is an Admin or a Regular User.

Screenshot 2 — Admin Login and Main Menu



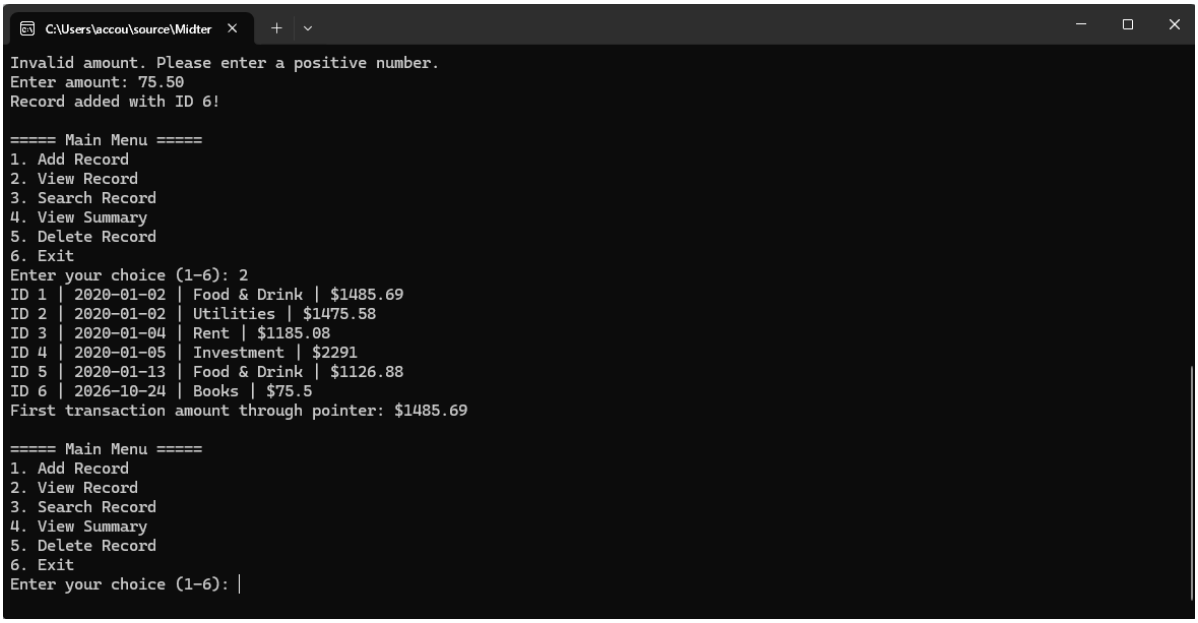
The user types "A" to log in as Admin. The program confirms the login and shows the Main Menu with all six options.

Screenshot 3 — Add Record (with Input Validation)



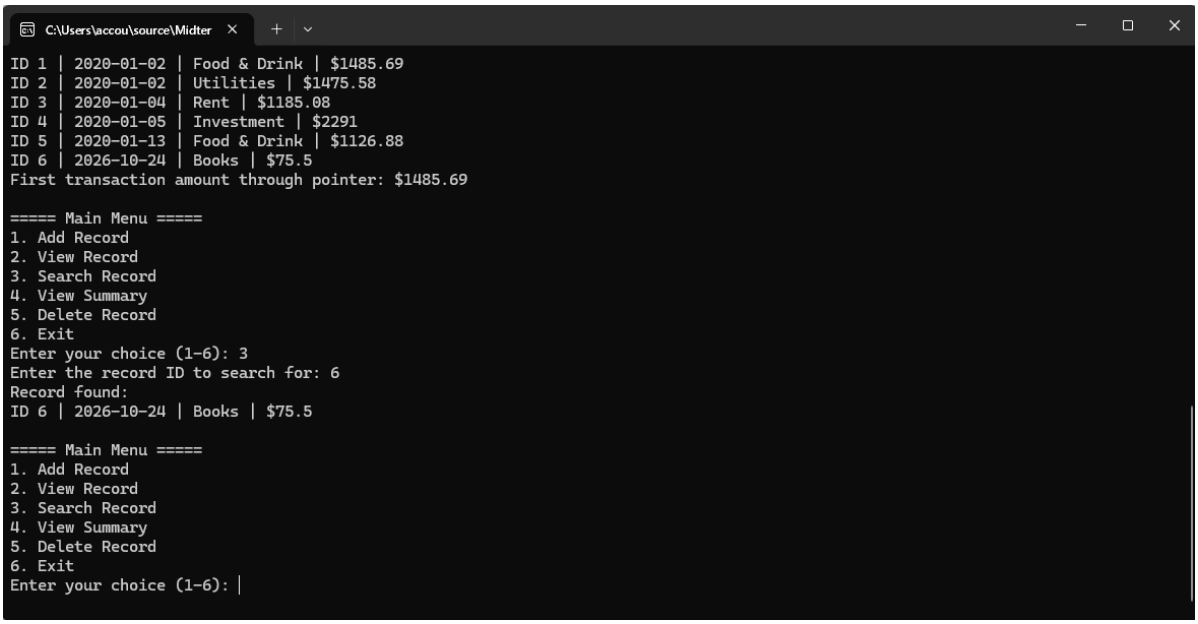
Choice 1 is selected to add a new record. The date "13/45/2026" is rejected for using the wrong format. An empty category is rejected. An amount of "-50" is rejected for not being positive. After fixing each mistake, the record is finally added successfully as ID 6.

Screenshot 4 — View Record



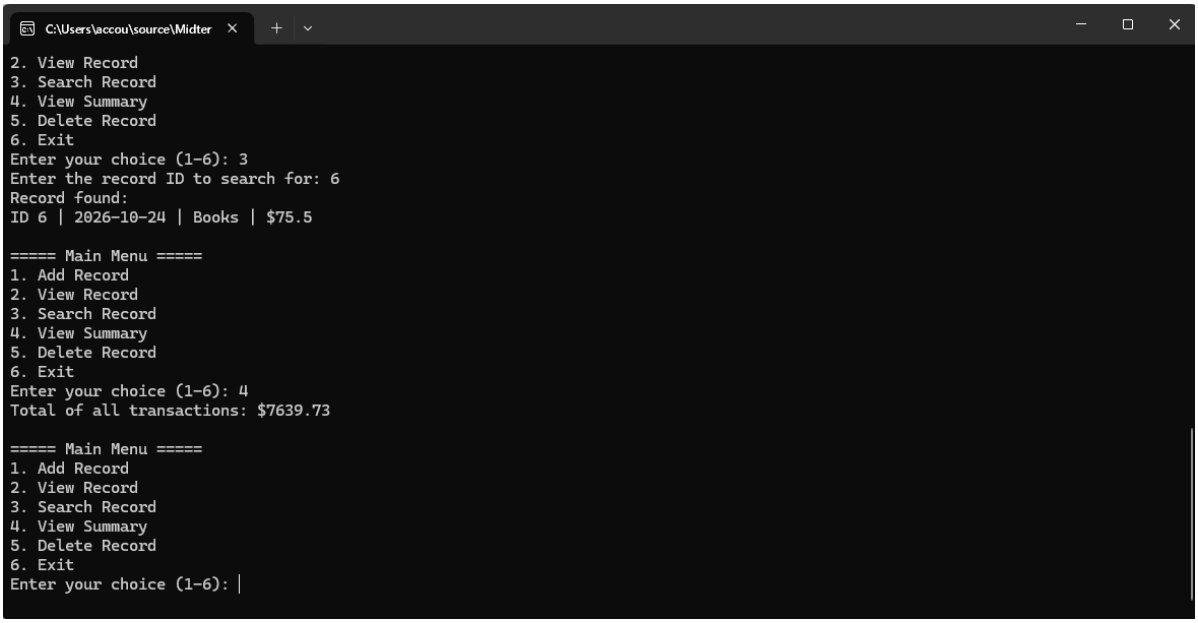
Choice 2 lists every saved record, including the new record (ID 6). The last line shows the first transaction's amount read through a pointer.

Screenshot 5 — Search Record (Binary Search)



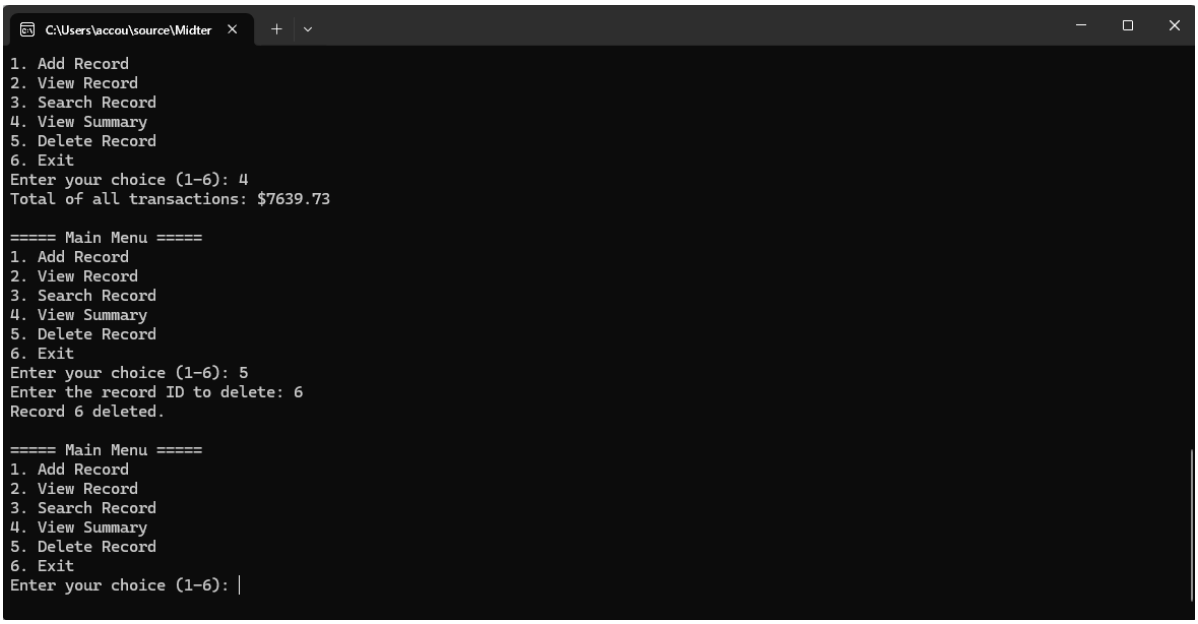
Choice 3 is selected and ID 6 is searched for. The binarySearchByID() function finds and displays the matching record.

Screenshot 6 — View Summary



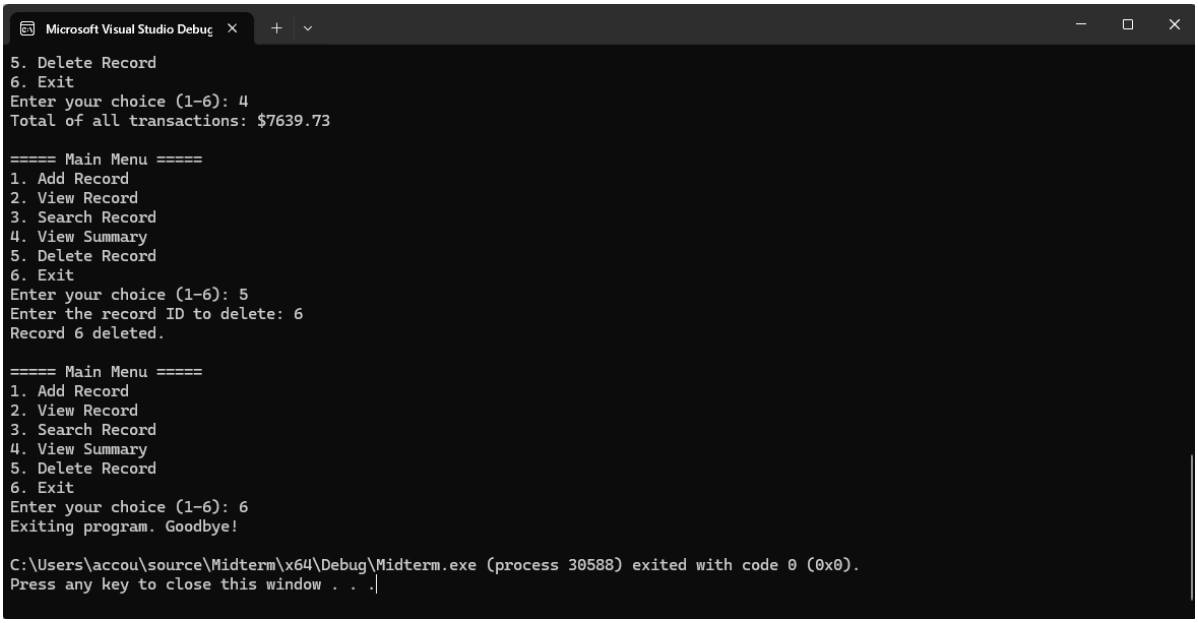
Choice 4 adds up every transaction amount and displays the total, $7639.73.

Screenshot 7 — Delete Record (Admin Only)



Choice 5 is selected and record ID 6 is deleted using binary search to find it first, then the record is removed and the file is saved again.

Screenshot 8 — Exit



Choice 6 ends the program. It prints "Exiting program. Goodbye!" and the program closes cleanly.